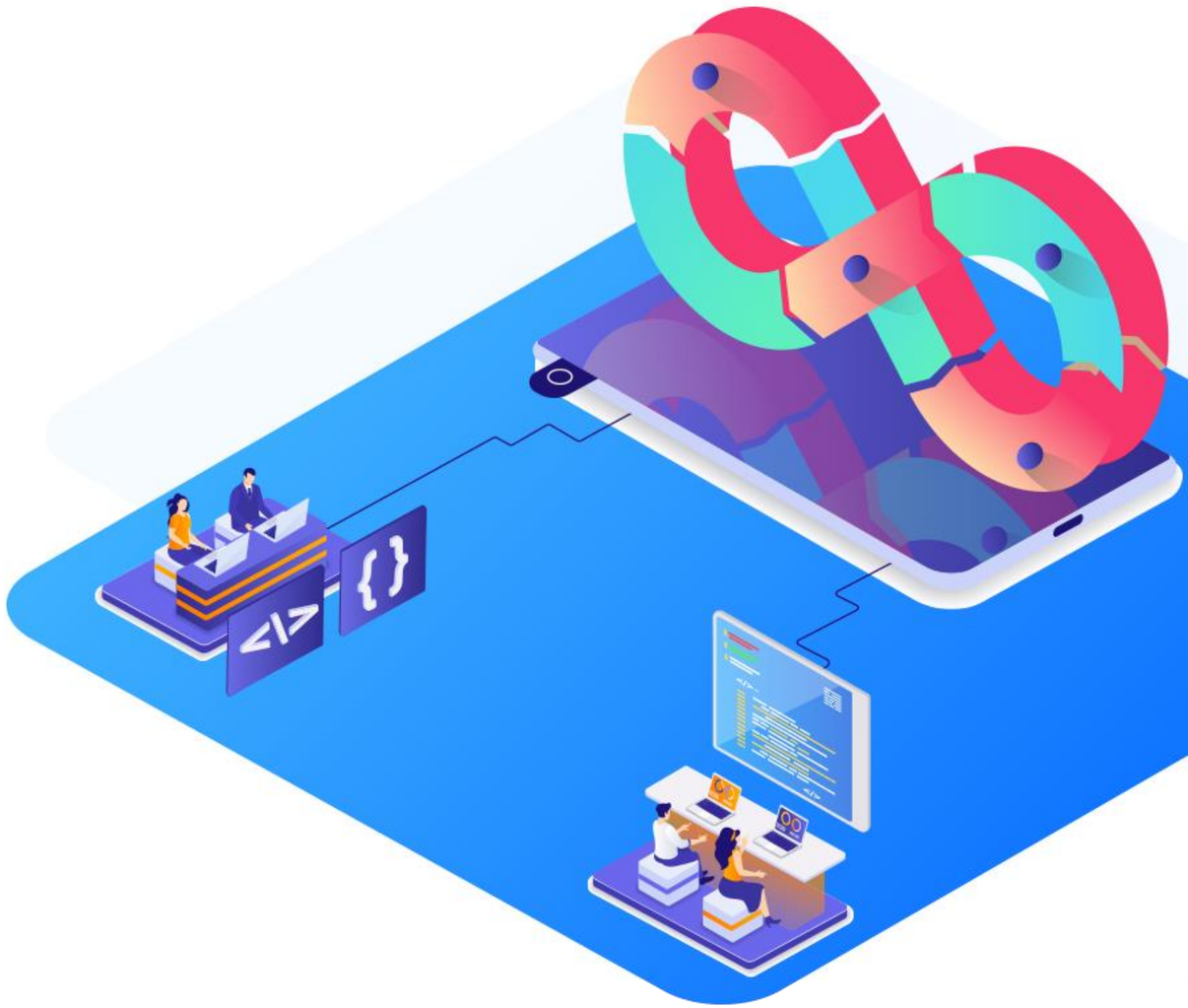
DevOps on AWS



Infrastructure as Code in AWS

Learning Objectives

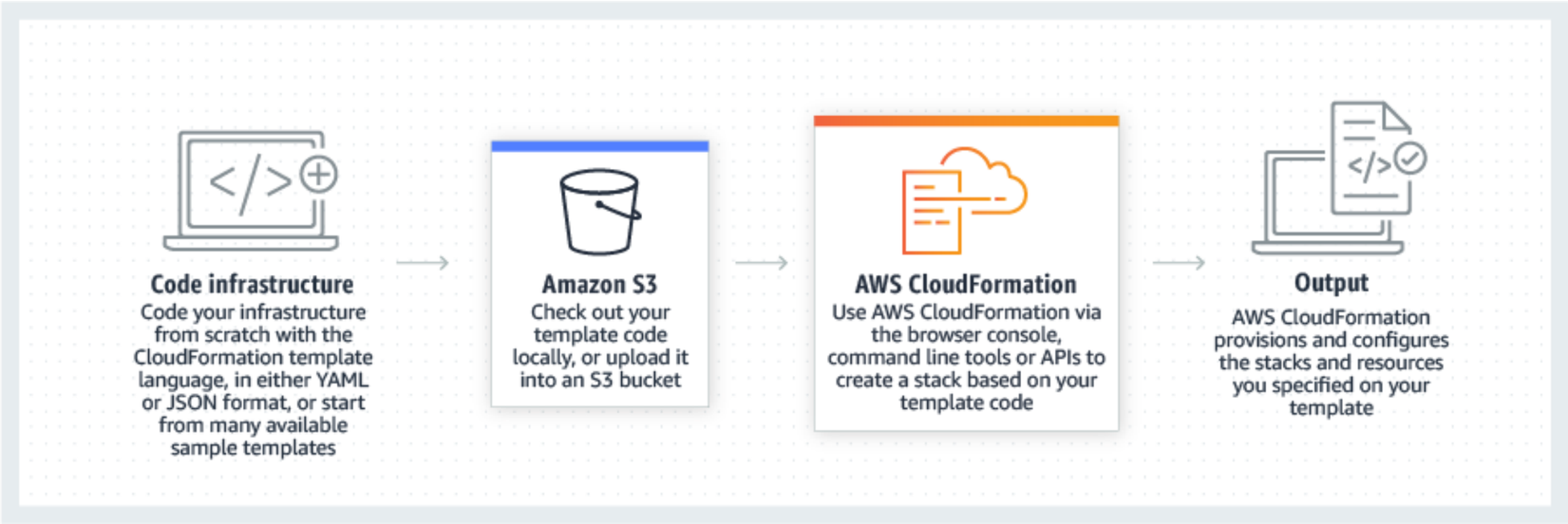
By the end of this lesson, you will be able to:

- Develop proficiency in building scalable cloud environments by using AWS CloudFormation
- Implement CloudFormation templates to integrate with various AWS services, improving workflow automation
- Create a basic CloudFormation template for launching a virtual machine (VM)
- Configure CloudFormation templates to meet specific infrastructure needs across different environments
- Explain the structure of CloudFormation templates, including the sections such as Resources, Parameters, Mappings, and Conditions

Introduction to CloudFormation

Introduction to CloudFormation

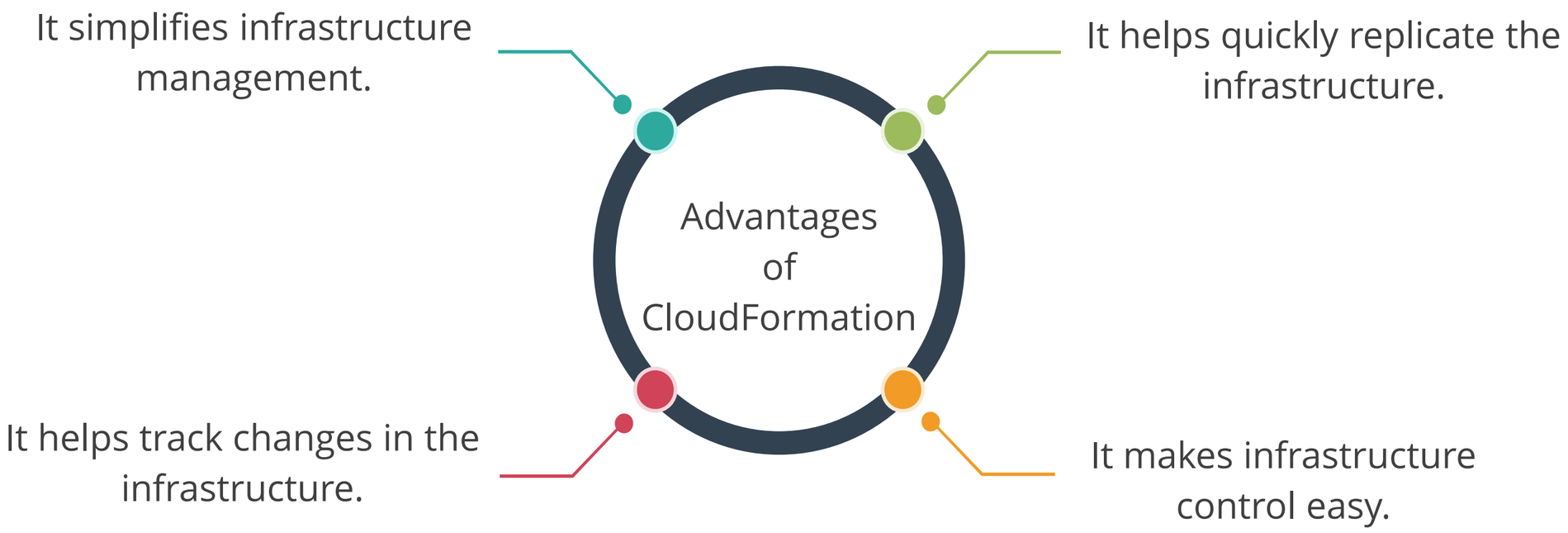
AWS CloudFormation is a service that helps an end-user automate AWS services by creating templates.



The CloudFormation interprets the template and makes the API calls to create the resources.

Source: https://aws.amazon.com/cloudformation/

Introduction to CloudFormation



Writing Infrastructure as Code Templates Using CloudFormation

Templates and Stacks in CloudFormation

Templates and **Stacks** are important elements of CloudFormation.

A **template** is a JSON or YAML-formatted text file.

A **stack** is a collection of AWS resources that a user can manage as a single unit.

Types of Templates

Template in JSON format:

```
{
  "AWSTemplateFormatVersion" : "version date",

"Description" : "JSON string",
"Metadata" : {
    template metadata },
"Parameters" : {
    set of parameters },
  "Rules" : {
    set of rules },
"Mappings" : {
    set of mappings },
"Conditions" : {
    set of conditions },
"Transform" : {
    set of transforms},
"Resources" : {
    set of resources  },
 "Outputs" : {
    set of outputs  }
}
```

Types of Templates

Template in YAML format:

```
AWSTemplateFormatVersion: "version date"
Description:
  String
Metadata:
  template metadata
Parameters:
  set of parameters
Rules:
  set of rules
Mappings:
  set of mappings
Conditions:
  set of conditions
Transform:
  set of transforms
Resources:
  set of resources
Outputs:
  set of outputs
```

Template Structure: Overview

The sections of the template are as follows:

01

Resources(required)

It includes the names and configurations of services to be added.

02

Outputs(optional)

This section includes values that are returned whenever users view their stack's properties.

03

Parameters(optional)

This includes input values that are provided while creating stacks.

04

Data tables(optional)

It includes static configuration values, such as AMI names.

Template Structure: Overview

The sections of the template are as follows:

05

Conditions(optional)

This includes actions to be taken based on conditional statements, such as equals.

06

Mappings(optional)

It has set values based on a region, such as AMIs.

07

Transform(optional)

This section includes snippets from outside the main template.

Template Structure: Overview

Template in JSON format:

```
{"AWSTemplateFormatVersion" : "version date",

"Description" : "JSON string",
"Metadata" : {
    template metadata },
"Parameters" : {
    set of parameters },
  "Rules" : {
    set of rules },
"Mappings" : {
    set of mappings },
"Conditions" : {
    set of conditions },
"Outputs" : {
    set of resources  },
 "Resources" : {
    set of outputs  }}
```

The template contains the following sections:

- Template version
- Description
- Metadata
- Parameter
- Mapping
- Conditions
- Output
- Resources

Resources Section: Example

Here is an example to show how to declare the AWS resources that need to be included in the stack:

```
"Resources": {
        "SQSQ44CP0": {
            "Type": "AWS::SQS::Queue",
            "Properties": {},
            "Metadata": {
                "AWS::CloudFormation::Designer": {
                    "id": "3f3e280c-2e3b-43d2-bdaa-9c9a529f6de8"
                }
            }
        },
        "S3B4XB7F": {
            "Type": "AWS::S3::Bucket",
            "Properties": {},
            "Metadata": {
                "AWS::CloudFormation::Designer": {
                    "id": "e5cbfee9-294b-4688-afd2-43274d15a8bc"
```

Parameter Section: Example

Here is an example to show how to specify input parameters for a template:

```
"Parameters‟ : {

 "InstanceTypeParameter":  {

"Type‟ :  "String",

 "Default‟  : "t2.micro",

"AllowedValues" : ["t2.micro", "m1.small",‟ml.large"],

"Description‟ : "Enter t2.micro, m1.small, or m1.large. Default is
t2.micro.‟
  }
}
```

Mapping Section: Example

Here is an example to show how to match a key to a corresponding set of named values:

```
"Mappings": {
    "Mapping01" : {
       "Key01" : {
         "Name":"Value01"
        }

      "Key02": {
         "Name" : "Value02"
         }
      "Key03": {
           "Name" : "Value03"
         }
     }
}
```

Condition Section

The properties of the condition section of the template are:

01

The condition statement is optional.

02

It contains statements that define the circumstances under which the resources are created.

Assisted Practice

Creating CloudFormation Stack for Launching VM **Duration: 15 Min.**

Problem Statement:

You have been assigned a task to create a new CloudFormation template to launch a new VM.

Outcome:

With this, CloudFormation will create a new EC2 instance with the specified instance type and key pair. The CloudFormation stack outputs will display the Instance ID and Public IP address of the newly created EC2 instance.

Note: Refer to the demo document for detailed steps:
01_Creating_CloudFormation_Stack_for_Launching_VM

Assisted Practice: Guidelines

Steps to be followed:

1. Log in to the AWS Console
2. Create a new CloudFormation template to launch a new VM

ASSISTED PRACTICE

Quick Check

Your team needs to deploy an application with EC2 instances, S3 buckets, and IAM roles using AWS CloudFormation. To handle frequent updates, which CloudFormation feature helps you manage changes in a scalable and repeatable way?

A. Stack sets

B. Change sets

C. Drift detection

D. AWS CloudTrail integration

Case Study on an E-commerce Company

Case Study: Problem Statement

An E-commerce company, EbuyEssence, manually set up and maintained its servers. As the company grew, the IT department struggled with:

- Ensuring consistent configurations across servers
- Deploying new environments rapidly
- Managing infrastructure changes and versioning

Case Study: Solution

01 **Choose an IaC Tool**: EbuyEssence decided to use CloudFormation due to its provider agnosticism, meaning it could work with multiple cloud providers.

02 **Version Control**: They stored their IaC files in a version control system (like Git), allowing them to track changes, revert to previous states if needed, and collaborate more efficiently.

03 **Define Infrastructure as Code**: The company started defining its infrastructure needs in configuration files, in which detailing required resources such as compute instances, storage, and networking components.

Case Study: Solution

04 **Continuous Integration and Continuous Deployment (CI/CD)**: EbuyEssence integrated IaC into their CI/CD pipeline, enabling automatic provisioning of required infrastructure for their application deployments.

05 **Monitoring and Auditing**: The company utilized tools that offer insights into the current state of their infrastructure, comparing it with the desired state defined in their IaC files.

Conclusion

Adopting Infrastructure as Code revolutionized EbuyEssence’s IT Operations, enhancing efficiency and scalability while reducing errors. This highlights IaC's pivotal role in today's cloud and DevOps contexts.

Assisted Practice

Launching an S3 Bucket Using CloudFormation **Duration: 15 Min.**

Problem Statement:

You have been assigned a task to launch an S3 Bucket using CloudFormation.

Outcome:

With this, you will successfully create a new CloudFormation template to launch a new S3 bucket, providing a scalable and automated way to manage S3 bucket creation.

Note: Refer to the demo document for detailed steps:
02_Launching_an_S3_Bucket_Using_CloudFormation

Assisted Practice: Guidelines

Steps to be followed:

1. Create a CloudFormation template

ASSISTED PRACTICE

Assisted Practice

Updating and Deleting S3 Bucket using CloudFormation **Duration: 10 Min.**

Problem Statement:

You have been assigned a task to perform updates and deletions on AWS CloudFormation stacks related to S3 buckets.

Outcome:

With this, you will successfully update and delete CloudFormation stacks related to S3 buckets, ensuring your infrastructure is managed efficiently and consistently.

Note: Refer to the demo document for detailed steps:
03_Updating_and_Deleting_S3_Bucket_using_CloudFormation

ASSISTED PRACTICE

Assisted Practice: Guidelines

Steps to be followed:

1. Update the CloudFormation stack
2. Delete operation on the S3 CloudFormation Stack

ASSISTED PRACTICE

Key Takeaways

- AWS CloudFormation is a service that helps an end-user automate AWS services by creating templates.
- A **stack** is a collection of AWS resources that a user can manage as a single unit.
- Data tables includes static configuration values, such as AMI names.
- Condition section contains statements that define the circumstances under which the resources are created.

Deploying EC2 Instances with Terraform and Custom VPC

Duration: 25 Min.

Project agenda: To demonstrate the process of launching an Amazon EC2 instance within a custom VPC using Terraform for automated infrastructure deployment and management.

Description: Your project manager wants a secure and isolated AWS setup for critical applications. You are tasked to deploy an EC2 instance in a custom VPC using Terraform, an Infrastructure as Code tool. This will ensure applications run in a controlled environment with automated deployment. Terraform scripts will define AWS resources, ensuring standardization and error reduction. Once executed, instances will be active within the VPC.

LESSON-END PROJECT

Deploying EC2 Instances with Terraform and Custom VPC

Duration: 25 Min.

Perform the following:

1. Create an IAM access key
2. Create a Terraform to launch the VM

Expected deliverables: A running EC2 instance on a custom VPC via Terraform scripts.

LESSON-END PROJECT

Thank you!